

MedImages.jl: A High-Performance, Differentiable Julia Framework for Medical Image Processing and Metadata Preservation

Jakub Mitura^{1*}, Divyansh Goyal¹, Michael C. Kreissl¹, Joanna Wybranska¹

¹ Division of Nuclear Medicine, Department of Radiology & Nuclear
Medicine, Faculty of Medicine, Otto von Guericke University Magdeburg,
39120 Magdeburg, Germany

* Corresponding author: jakub.mitura@med.ovgu.de
Full Postal Address: Leipziger Str. 44, 39120 Magdeburg, Germany
Telephone: 00493916713000

Abstract

Background and Objectives: Quantitative medical imaging, particularly in Nuclear Medicine, requires software that ensures both high-throughput computational performance and strict spatial metadata integrity. Existing frameworks often face a "two-language" bottleneck, bridging high-level APIs with compiled backends, which complicates automatic differentiation (AD) and can lead to metadata drift during preprocessing. We introduce MedImages.jl, a native Julia ecosystem for 3D/4D medical image analysis.

Methods: MedImages.jl implements a typed data structure that intrinsically binds voxel data to spatial metadata (origin, spacing, direction). Core geometric transformations are implemented as native, differentiable Julia kernels compatible with GPU acceleration (KernelAbstractions.jl) and SciML AD engines (Zygote.jl). The framework integrates HDF5 for high-throughput I/O and supports batched multimodality operations.

Results: In a 100-subject PET/CT preprocessing benchmark, MedImages.jl demonstrated a $7.2\times$ total turnaround speedup compared to standard Python-based caching pipelines. GPU-accelerated spatial transformations achieved up to $135\times$ speedups over established single-threaded C++ baselines like SimpleITK. We validated end-to-end differentiability through a spatial transformer network and multi-organ affine registration, and confirmed that Standardized Uptake Value (SUV) calculations match established clinical tools to double-precision accuracy.

Conclusions: MedImages.jl provides a performant, fully differentiable environment for medical image processing. By unifying spatial operations and metadata management within a single language, it facilitates large-scale clinical research and physics-informed deep learning without compromising clinical validity.

Keywords: Medical Imaging, Julia, Scientific Machine Learning, Nuclear Medicine, Differentiable Programming, Spatial Metadata

1 Introduction

The development of MedImages.jl is driven by the need to address four converging challenges in contemporary medical imaging pipelines, particularly within Nuclear Medicine workflows that rely heavily on functional data (e.g., PET, SPECT) and anatomical references (CT, MRI).

Challenge 1: The Volume of Modern Biobank Datasets. A single whole-body PET/CT examination involves high-resolution 3D volumes that must be co-registered, resampled, and quantitatively normalized. At biobank scale—encompassing thousands of subjects—preprocessing these massive volumetric datasets becomes a dominant bottleneck. In traditional hybrid environments (SimpleITK [1], MONAI [2]), I/O overhead and metadata synchronization account for a significant portion of processing time, leading to substantial delays in multimodal dataset preparation [3].

Challenge 2: Execution Speed and the "Two-Language" Barrier. Researchers often prototype in high-level languages like Python but must rely on compiled C++ backends for execution speed. This "two-language problem" complicates deployment and limits code inspectability. Julia [4] addresses this by compiling directly to efficient machine code via LLVM JIT, allowing spatial transformations to be executed rapidly across both CPU and GPU backends using native implementations rather than wrapped binaries.

Challenge 3: Differentiability and the Fragmented SciML Landscape. Emerging Scientific Machine Learning (SciML) paradigms require pipelines where every computational step is differentiable. While Python offers powerful differentiable libraries (e.g., PyTorch, JAX, TensorFlow), they often function as "walled gardens." Composing a pipeline requires forcing the entire ecosystem to commit to a single framework, creating friction when integrating third-party tools or classical numerical solvers [5,6]. Julia addresses this structural fragmentation by solving the "expression problem" through multiple dispatch, allowing native code across different packages to seamlessly interoperate [7]. Because Julia's automatic differentiation engines (e.g., Zygote.jl [8], Enzyme.jl) operate at the compiler level, they can differentiate through arbitrary code, including complex differential equation solvers and GPU kernels, without requiring users to rewrite logic into a specific tensor framework. MedImages.jl introduces a critical tool to make this unique, highly composable SciML ecosystem widely accessible to medical imaging researchers.

Challenge 4: Metadata Management and Physical Interpretation. Unlike standard computer vision, medical images possess a strict "physical interpretation." A medical image is a grid of points occupying a physical region defined by an origin, spacing, and direction. The transition from medical image formats (DICOM, NIfTI) to raw numerical arrays for deep learning often strips this spatial context. This risks "metadata drift"—a potentially dangerous misalignment between voxel data and its spatial coordinates during preprocessing or augmentation.

MedImages.jl provides a unified, GPU-accelerated, and differentiable imaging framework designed explicitly to address these four challenges. The

structure of this paper, including the methodology and results, is organized around validating solutions to these core axes.

1.1 Architectural Foundations: The Coordinate System and Metadata

Addressing Challenge 4 (Metadata Management) requires ensuring that data remains defined within a physical coordinate system. To accurately integrate functional data with anatomical data across different spatial resolutions, software must rigorously maintain the mapping between discrete voxel indices and physical patient space. This mapping from voxel coordinates $\mathbf{v} = [i, j, k, 1]^\top$ to continuous world coordinates $\mathbf{p} = [x, y, z, 1]^\top$ relies on a 4×4 homogeneous affine transformation matrix M :

$$\mathbf{p} = M\mathbf{v}, \quad M = \begin{bmatrix} RS & T \\ \mathbf{0} & 1 \end{bmatrix} \quad (1)$$

where R is the 3×3 rotation matrix, S is a diagonal scaling matrix (voxel spacing), and T is the translation vector (origin). MedImages.jl ensures that any spatial operation $\mathcal{T}$ applied to the voxel grid is mathematically coupled with an update $M_{new} = M_{old} \times M_{\mathcal{T}}$, preventing spatial desynchronization.

Furthermore, quantitative nuclear medicine relies on precise temporal and clinical metadata. The Standardized Uptake Value (SUV), a critical metric for PET/CT, is calculated as:

$$SUV = \frac{C(t)}{ID/BW} \quad (2)$$

where $C(t)$ is the decay-corrected radioactive concentration, ID is the injected dose, and BW is patient body weight. Discarding parameters such as *Acquisition Time* or *Patient Weight* compromises the quantitative utility of the image. MedImages.jl intrinsically protects these fields within its data structures.

2 Methods

Software Architecture

MedImages.jl implements a metadata paradigm inspired by the Brain Imaging Data Structure (BIDS) standard, organizing and validating clinical metadata rigorously. The core **MedImage** struct enforces metadata integrity through Julia’s type system. Unlike array-based deep learning workflows where converting to tensors discards spatial context, MedImages.jl intrinsically binds metadata (origin, spacing, direction, patient ID, acquisition timestamp) to the structural definition of the medical image.

ITK (C++) is used via wrapper primarily for I/O robustness (NIfTI, DICOM parsing), while spatial transformations are implemented natively in Julia. This ensures: (1) **Differentiability**—operations integrate with Zygote.jl/Enzyme.jl; (2) **Composability**—seamless integration with Lux.jl and DifferentialEquations.jl.

To support high-throughput analysis of multimodal studies, MedImages.jl introduces **BatchedMedImage**, which stores 4D voxel data (x, y, z, batch) . Crucially, the spatial metadata (origin, spacing, direction) is vectorized. This design allows each 3D slice within the batch to maintain unique spatial

properties while residing in a unified 4D tensor, facilitating efficient batched operations where identical geometric transformations can be dispatched to the GPU in a single pass. To minimize I/O bottlenecks for large datasets, the framework supports HDF5-based storage.

Code Example: Typical Workflow

The following snippet demonstrates loading a multimodal pair, rotating them simultaneously using the batched structure, and computing the SUV:

```
using MedImages

# Load PET and CT (DICOM metadata is preserved)
pet = load_image("patient_1/pet.dcm")
ct = load_image("patient_1/ct.nii")

# Create a batched tensor to process simultaneously
batch = create_batched_medimage([pet, ct])

# Rotate both volumes by 45 degrees around Z-axis (GPU supported)
rotated_batch = rotate_mi(batch, 45.0)

# Extract SUV statistics for the PET scan using an ROI mask
suv_stats = calculate_suv_statistics(rotated_batch[1], roi_mask)
println("Mean SUV: ", suv_stats.mean_suv)
```

Functionalities and I/O Robustness

MedImages.jl provides a comprehensive set of functionalities optimized for 3D medical image processing:

- **Robust I/O:** Loading of various medical image formats (NIfTI, DICOM) using `ITKIOWrapper`.
- **Native Julia Transformations:** Functions to modify spatial properties of 3D volumes (`rotate_mi`, `translate_mi`, `scale_mi`, `resample_to_spacing`, `crop_mi`, `pad_mi`).
- **Resample to Target:** Resamples a moving image to match the spatial geometry of a reference image.
- **GPU Acceleration:** All transformations transparently support GPU execution via `KernelAbstractions.jl`.
- **HDF5 Support:** Fast loading and saving of 3D images using HDF5, speeding up I/O for intermediate processing and reducing the "loading penalty" common in deep learning pipelines.
- **Autodifferentiation:** All spatial transformations have well-defined gradients via `Zygote.jl` and `Enzyme.jl`.
- **Orientation Management:** Robust tools to handle and convert between neuroimaging coordinate systems.

Experimental Setup: Evaluating the Four Challenges

To systematically evaluate the framework against the four core challenges, several experimental pipelines were designed.

Evaluating Volume and Speed

For large-scale studies requiring standardized preprocessing of heterogeneous medical volumes, we developed a fused affine preprocessing pipeline that collapses multiple spatial transformations—orientation normalization, spacing resampling, and spatial centering—into a single 4×4 affine matrix. This matrix is precomputed on the CPU from spatial metadata, then applied via a custom `KernelAbstractions.jl` kernel with unrolled trilinear interpolation on the GPU.

To evaluate the framework under high-throughput production loads (Challenge 1: Volume), we benchmarked a 100-case preprocessing pipeline on the autoPET dataset [3] against MONAI’s `PersistentDataset` mechanism. A 100-case dataset was selected to represent a typical multimodality workflow bottleneck: physically disparate modalities (PET and CT) must be loaded and strictly aligned to a common spatial reference frame—meaning they share the identical voxel spacing, direction cosines, and origin—so they can be safely concatenated into a single multi-channel tensor for deep learning ingestion.

To assess raw execution speed (Challenge 2: Speed), we benchmarked isolated transformations across multiple scales. `SimpleITK` benchmarks were executed strictly on the CPU, as it does not natively support GPU acceleration for these geometric filters. Conversely, `MedImages.jl` and MONAI benchmarks utilized GPU execution to demonstrate the high throughput enabled by natively fused kernels. All experiments described are open-source and reproducible via the codebase provided in this repository.

Evaluating Differentiability (SciML Integrations)

To validate end-to-end differentiability (Challenge 3), we implemented three paradigms:

1. Learned Inverse Rotation (Differentiable Geometric CNN):

A neural network learns to predict and invert unknown 3D rotations applied to volumetric data. To avoid confusion with modern attention-based architectures, we clarify that this uses a standard 3D Convolutional Neural Network (CNN) as a spatial regressor. The CNN predicts three Euler angles, which are converted to an affine matrix to generate a differentiable sampling grid for `MedImages.jl`’s `interpolate_pure` function.

2. Organ-Specific Affine Registration:

A 3D CNN predicts per-organ affine parameters. The loss computation is a single GPU kernel fusing affine transformation, a barycenter distance penalty, and trilinear interpolation loss, differentiated via `Enzyme.autodiff`.

3. Quantitative Dosimetry Refinement via Universal Differential Equations:

A pipeline designed to correct approximate 3D SPECT reconstructions into high-accuracy, patient-specific dose maps. By embedding neural residual networks within a mechanistic differential equation [9], this approach aims to emulate the accuracy of full Monte Carlo (MC) tracking while maintaining the interpretability essential for medical practice.

The clinical imperative for high-accuracy dosimetry has never been more acute as radiation oncology transitions toward personalized interventions

like Targeted Radionuclide Therapy (TRT) and Transarterial Radioem-
bolization (TARE). Historically, dosimetry relied on simplified Medical
Internal Radiation Dose (MIRD) formalisms that assumed uniform activ-
ity distribution within source organs. While standard clinical methods
have transitioned to voxel-based S-values and convolution kernels, they
often assume a homogeneous medium, leading to significant errors at tis-
sue interfaces or highly heterogeneous regions (e.g., lungs). While Monte
Carlo techniques solve this by tracking stochastic particle interactions, their
massive computational burden restricts their clinical utility.

To address this, our pipeline models dose deposition as a parameterized
dynamical system. Unlike purely data-driven networks that act as "black
boxes," UDEs explicitly satisfy the need for interpretability by treating
dose refinement over integration steps as an Initial Value Problem (IVP):

$$\frac{dD}{dt} = f_{\text{mechanistic}}(S, C, D) + \text{NN}(S, C, D; \theta) \quad (3)$$

Here, S is the measured SPECT activity, C is the CT-derived tissue density,
and D is the cumulative dose. The $f_{\text{mechanistic}}$ term provides a strong induc-
tive bias by hard-coding well-understood primary physical laws—specifically,
local energy deposition modeled as proportional to activity and local tissue
density. This constraint ensures data efficiency and safety, as the model
produces physically plausible results even in regions poorly represented in
training data.

The neural residual network, NN_{θ} , is the "universal" component embed-
ded directly within the differential equation to learn the highly complex,
non-local scattering phenomena (e.g., Compton and Rayleigh scattering)
and secondary emissions (e.g., Bremsstrahlung) that cause energy to be
deposited at a distance from the source activity. By training against high-
fidelity MC ground truths, the network learns to approximate the residual
difference between the simple mechanistic physics model and the true dose
distribution. To ensure strict differentiability across the entire 5D tensor
without encountering Zygote.jl array mutability tracking limitations, this
IVP is solved using a custom, out-of-place manual 5-step Euler integration
loop rather than standard `SciMLSensitivity` adjoints.

3 Results

To demonstrate that `MedImages.jl` resolves the core computational and
metadata challenges, our results are structured along the axes defined in
the Introduction.

Challenge 1: Volume – Scaling to 100 Cases (HDF5 vs. Caching)

To evaluate the framework under high-throughput loads, we benchmarked
a 100-case preprocessing pipeline on the `autoPET` dataset. The core com-
putational task was a multi-modal alignment pipeline, resampling highly
heterogeneous PET and CT volumes to a standardized $512 \times 512 \times 512$
isotropic grid.

In clinical practice, "alignment" implies mathematically mapping the
voxel grid of a functional image (e.g., PET, which often has lower resolution
and varying spatial origins) to the exact coordinate space of an anatomical

reference image (e.g., CT). This requires generating a continuous interpolation grid based on the source metadata (origin, spacing, and direction cosine matrix) and resampling the raw voxel intensities. This alignment is the foundational prerequisite for modern multimodality analysis; Convolutional Neural Networks (CNNs) require multi-channel tensors where pixel (i, j, k) in the PET channel corresponds to the exact same physical tissue location as pixel (i, j, k) in the CT channel. Due to the massive scale of 3D modalities, this continuous spatial resampling is often the dominant bottleneck in deep learning pipelines.

For a fair comparison of computational overhead in this high-volume scenario, we evaluated performance for high-frequency training loops (Epochs 2+), assuming data has been initially processed and cached. We compared MedImages.jl utilizing HDF5 against MONAI’s `PersistentDataset` mechanism. It should be noted that this compares different data persistence strategies—HDF5 vs. Pickle/Pt-based caching—as much as the underlying framework performance.

Table 1: Definitive Performance Comparison (100 Subjects, Epochs 2+)

Phase (per Subject)	MedImages (HDF5)	MONAI (PersistentDataset)
Stage 1: Load from Cache	40 ms	150 ms
Stage 2: Transform (Compute)	50 ms	500 ms
Total Turnaround	90 ms	650 ms

Overall, MedImages.jl achieved a $7.2\times$ total turnaround speedup compared to the MONAI `PersistentDataset` baseline.

Challenge 2: Speed – GPU Acceleration Benchmarks

MedImages.jl was benchmarked on CPU and GPU backends (NVIDIA RTX 3090, Intel Core Ultra 9 285K). For $256 \times 256 \times 128$ volumes, MedImages.jl on GPU achieved significant speedups compared to SimpleITK on CPU:

- **Resampling (Nearest Neighbor):** $115\times$ speedup vs CPU, $17\times$ vs SimpleITK.
- **Orientation Changes:** $71\times$ speedup vs CPU, $31\times$ vs SimpleITK.
- **Fused Affine Transformation:** $135\times$ speedup vs CPU, $8.1\times$ vs SimpleITK.

Regarding the Fused Affine Pipeline defined in the methods, benchmarking on 100 subjects resampled to a 512^3 target grid demonstrated per-subject GPU kernel times of under 200 ms, confirming the pipeline’s suitability for biobank-scale processing.

Challenge 3: Differentiability and SciML Validation

The Julia ecosystem is uniquely positioned for SciML because its automatic differentiation frameworks (Zygote.jl, Enzyme.jl) can differentiate through arbitrary native code, including custom spatial transformations. Nuclear medicine imaging is inherently quantitative, but its key readouts (e.g., SUV, time–activity curves, absorbed dose) are highly sensitive to geometric pre-processing such as resampling, rotation, and registration. In practice, these operations must often be integrated into AI pipelines for motion correction,

longitudinal alignment, and learned preprocessing, where transform parameters are optimized jointly with network weights. However, many commonly used imaging backends execute spatial resampling outside the automatic differentiation graph, preventing end-to-end training through geometric steps [9]. To demonstrate that MedImages.jl enables fully differentiable volumetric spatial operations suitable for nuclear medicine workflows, we implement several SciML validation tests.

Differentiability Verification Tests

To rigorously validate correctness, we verified that Zygote.jl returns non-trivial, structurally correct gradients across all spatial transforms. For identity-preserving operations (`translate_mi`, `pad_mi`), the gradient propagates as `1` without corruption. For spatially selective operations (`crop_mi`), the gradient is 1 inside the cropped region and 0 outside. For the fused affine interpolation kernel, we validated CPU/GPU gradient consistency to within 10^{-4} .

Learned Spatial Transformations

In the Learned Inverse Rotation experiment, end-to-end differentiability of the geometric pipeline was confirmed using synthetic single-case batched configurations. Over 20 training iterations, the CNN successfully optimized the predicted Euler angles, consistently reducing the mean squared reconstruction error (MSE) between the de-rotated output and the original target by over 65%. This demonstrates that differentiable augmentation and dynamic learned preprocessing—which are often difficult or rare in standardized imaging pipelines [10]—can be seamlessly achieved using native Julia operations.

Similarly, the Organ-Specific Affine Registration pipeline converged using the custom fused GPU loss kernel. The ability of Enzyme.jl to compute exact reverse-mode gradients through the highly branched KernelAbstractions.jl code validated that complex spatial regularizations can be trained natively on the GPU without hand-writing CUDA adjoints.

Quantitative Dosimetry Refinement Performance

The UDE-based dosimetry refinement pipeline successfully corrected approximate SPECT reconstructions into high-accuracy equivalents. The Universal Differential Equation model achieved steady convergence, reaching an MSE of 4.14×10^{-4} .

Crucially, the interpretability of the UDE approach is profound: it successfully isolates the known mechanistic physics (primary deposition) from the unknown statistical noise (complex scattering). This allows clinical physicists to independently verify the contribution of the mechanistic term, ensuring that primary dose components adhere to established standard models. Simultaneously, the neural residual robustly corrects secondary scattering artifacts rather than hallucinating primary energy distributions.

This level of precision is paramount in therapeutic nuclear medicine, where ablative doses are delivered to tumor tissue (e.g., Hepatocellular Carcinoma via Yttrium-90 microspheres) while attempting to spare healthy organs. Standard dose kernels fail to account for the heterogeneous density of tissue interfaces and "leakage" across boundaries. By learning these non-local interactions from MC data, the UDE framework generates a

more accurate spatial representation of the dose distribution. This enables clinicians to confidently rely on critical volumetric coverage metrics, such as Dose-Volume Histograms (DVHs) to calculate D95 (minimum dose covering 95% of the target volume) and D70, predicting pathological response rates accurately without the prohibitive computational costs of full Monte Carlo execution.

Challenge 4: Metadata Management and Clinical Fidelity

We verified the quantitative accuracy of MedImages.jl by comparing its Standardized Uptake Value (SUV) calculations against the gold standard SimpleITK. Due to Julia’s high-precision handling of temporal and clinical metadata, MedImages.jl achieves identical results for SUVbw (Body Weight) calculations, matching SimpleITK’s output up to 10^{-14} in double precision. This ensures that the massive speedups achieved through GPU acceleration do not compromise clinical validity, a fact verifiable through the open-source benchmarks in this repository.

Furthermore, we conducted an end-to-end consistency experiment on the `autoPET` dataset [3] to verify that metadata preservation and quantitative fidelity are maintained throughout complex spatial transformation pipelines. The evaluation involved a multi-modal sequence consisting of (1) converting PET intensities to SUV, (2) resampling the PET and label masks to the native CT target space, (3) changing the spacing to a 2.0 mm isotropic grid, (4) applying a 45-degree rotation around the Z-axis, and (5) applying a 10-voxel translation along the X-axis. For the segmentation masks, nearest-neighbor interpolation was strictly applied to prevent boundary blurring, while linear interpolation was utilized for the continuous PET/CT intensities.

Quantitative analysis of the Mean SUV and total Lesion Volume inside the mask demonstrated remarkable consistency before and after these compound transformations. The original Mean SUV was 3.4913 with a volume of 52.55 cm³, and the final transformed volume exhibited a Mean SUV of 3.4660 and a volume of 51.86 cm³. The minor deviations in Mean SUV (0.73%) and Volume (1.32%) are fundamentally attributable to grid resampling artifacts and interpolation rather than underlying metadata deterioration. Figure 1 visually illustrates the multi-modal alignment before and after the full transformation pipeline.

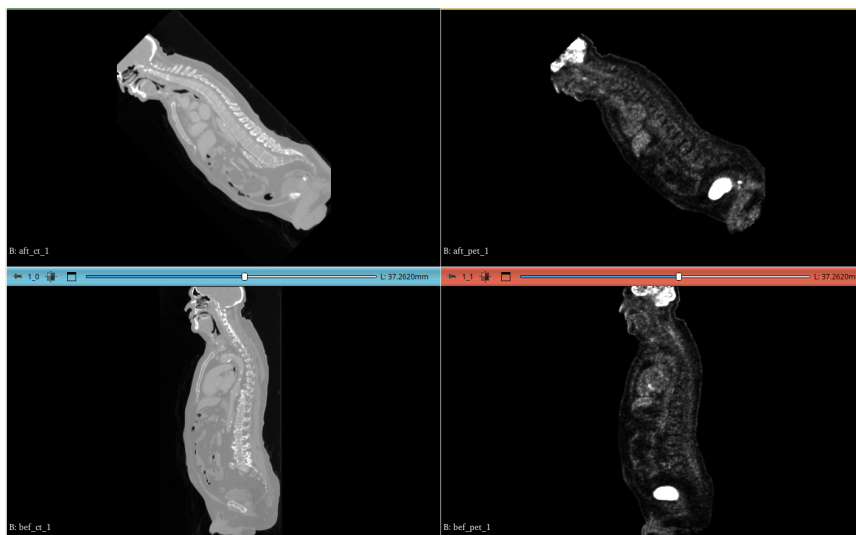


Figure 1: **Quantitative Consistency Under Compounded Spatial Transformations (Sagittal Views)**. The four-panel composite shows sagittal slices of the same subject before and after a sequence of spatial transformations (isotropic 2.0 mm resampling, 45° Z-rotation, X-translation): **Top-left**: CT after transformations; **Top-right**: PET/SUV after transformations; **Bottom-left**: CT in original native space; **Bottom-right**: PET/SUV in original native space. MedImages.jl preserves spatial alignment and clinical metadata throughout, with Mean SUV (3.4913 vs 3.4660) and gross lesion volume (52.55 cm³ vs 51.86 cm³) varying by < 1.5% across the full pipeline.

Illustrative Example: Multi-Modal Batched Processing in Theranostics

Theranostic radioligand therapy workflows integrate longitudinal patient imaging acquired at different time points, on different devices and reconstruction pipelines—e.g., [¹⁷⁷Lu]Lu-PSMA SPECT with and without attenuation correction (AC/NAC), voxel-wise absorbed dose maps, and CT—where each modality retains its own voxel grid, datatype, and spatial metadata (spacing, origin, orientation).

To demonstrate the capability of `BatchedMedImage` in handling complex, multi-modal datasets typical for theranostics workflows, we processed a dataset of four typical examinations, each containing a distinct imaging modality:

¹⁷⁷Lu Lu-PSMA SPECT with attenuation correction (AC)

¹⁷⁷Lu Lu-PSMA SPECT without attenuation correction (NAC)

¹⁷⁷Lu Lu-PSMA Dosemap (voxel-wise absorbed dose)

1. Computed Tomography (CT) for anatomical reference

The images were loaded into a single `BatchedMedImage` object, preserving their individual spatial metadata and voxel types. A batched rotation

transformation was then applied simultaneously to all four modalities. Figure 4 illustrates the results, showcasing the original axial slices (left) and the rotated output (right).

The ability to apply identical geometric transformations across functional (SPECT, Dosemap) and anatomical (CT) data in a single operation ensures perfect spatial alignment is maintained during augmentation or registration tasks, which is critical for accurate dosimetry and treatment planning.

Figure 2: *
A: Original

Figure 3: *
B: Rotated

Figure 4: **Simultaneous Batched Rotation of Multi-Modal Theranostic Data.** Panel A displays the original axial slices of four distinct modalities stored within a single `BatchedMedImage` structure: (Top-Left) ^{177}Lu Lu-PSMA SPECT AC, (Top-Right) ^{177}Lu Lu-PSMA SPECT NAC, (Bottom-Left) ^{177}Lu Lu-PSMA Dosemap, and (Bottom-Right) CT. Panel B shows the result after applying a single `rotate_mi` operation to the batch. The spatial integrity and alignment between the functional distribution (SPECT/Dose) and anatomical structure (CT) are preserved during the transformation.

4 Discussion

As biomedical imaging increasingly embraces complex multimodal, multidimensional, and longitudinal studies, alongside scientific machine learning (SciML), the interoperability and performance of data management frameworks become critical. Our results indicate that MedImages.jl's metadata handling paradigm, combined with seamless integration into the Julia scientific ecosystem, provides a highly performant and differentiable framework that systematically addresses the three core challenges outlined in the Introduction.

Addressing Challenges 1 and 2 (Volume and Speed): The Advantage of Native Fused Kernels

Integration of metadata within the `MedImage` structure directly translates into performance gains observed in our benchmarks. The $7.2\times$ turnaround advantage observed in high-frequency training loops is primarily attributed

to MedImages.jl’s integration with HDF5 – which bypasses the serialization costs inherent in many Python caching mechanisms – and its single-pass Fused Affine kernel. By mathematically collapsing the entire spatial transformation stack (normalization, resampling, centering, etc.) into a single CUDA execution, MedImages.jl eliminates the memory traffic and synchronization overhead associated with sequential operation queues, providing a highly efficient solution for biobank-scale deployments.

Addressing Challenge 2 (Metadata Management): Comparing Metadata Architectures

Modern Python frameworks like MONAI [2] have introduced sophisticated mechanisms to couple metadata with image arrays. MONAI utilizes a MetaTensor structure, appending a metadata dictionary to the PyTorch Tensor. A similar design pattern appears in TorchIO [?], which prioritizes convenient multimodal coordination but still builds its core processing around external imaging backends. TorchIO approaches the problem through a hierarchical Subject class system. Although effective for multimodal synchronization, TorchIO is primarily a manager of third-party backends like SimpleITK or NiBabel. This the dependency chain maintains a layer of indirection that can lead to memory management issues in large-scale training [?].

In contrast, MedImages.jl leverages Julia’s type system to bind metadata directly within the `MedImage` struct. As summarized in Table 2, by removing the runtime overhead associated with dictionary-based metadata access, this approach contributes to the improved performance observed in our benchmarks. Furthermore, the native implementation of core geometric transformations in Julia enables MedImages.jl to maintain the automatic differentiation graph across processing steps. This is particularly relevant for imaging workflows that incorporate scientific machine learning, where gradient information may be lost when operations are passed to non-differentiable compiled backends.

Table 2: Comparative Metadata Architectures in Modern Frameworks

Framework	Metadata Storage Mechanism	Preservation Strategy	Potential Interoperability Constraints
SimpleITK	Internal C++ Struct	Encapsulated in <code>itk::Image</code>	User-dependent retention (often discarded upon conversion to NumPy).
MONAI	MetaTensor (Dict-based)	Metadata dictionary moves with tensor	Dictionary lookup overhead; potential orientation flip during ITK interoperability.
TorchIO	Subject Class Attributes	Multi-modal synchronization	Potential memory buildup during extensive transformation histories.
MedImages.jl	Typed Julia Struct	Explicit binding within struct	N/A (Native Julia stack).

The Primacy of Clinical and Temporal Metadata

The lack of standardization in medical imaging preprocessing pipelines can pose a reproducibility challenge [11]. In such settings, metadata drift, defined as the decoupling of spatial or clinical metadata from voxel data, may introduce clinically relevant errors. For example, it can lead to displacement of treatment margins in radiotherapy or compromise quantitative interpretation in PET/CT when decay-related timing information is lost [11]. Quantitative nuclear medicine is particularly dependent on the integrity of clinical and temporal metadata, as standardized uptake value (SUV) calculations explicitly require accurate injection and acquisition times. In theranostic workflows, this requirement must be maintained while applying identical geometric transformations across multiple co-registered modalities, such as ^{177}Lu Lu-PSMA SPECT AC/NAC, absorbed-dose maps, and CT.

MedImages.jl addresses this by design: the `BatchedMedImage` abstraction allows single-call, batched geometric transforms to be applied consistently across modalities without manual bookkeeping. Our cross-platform validation confirms that the SUV conversion factors extracted by MedImages.jl achieve mathematical identity with established SimpleITK benchmarks to a precision of 10^{-14} . This numerical equivalence ensures that researchers transitioning to Julia can maintain absolute continuity with legacy clinical findings while gaining the performance required for massive biobank-scale analysis.

Addressing Challenge 3 (Differentiability): Navigating the Framework Ecosystem

We do not advocate replacing established tools wholesale. Rather, we recommend choosing the framework that best matches the task at hand, especially when differentiability and complex scientific computing are core requirements.

When established Python frameworks remain the pragmatic choice. For teams with substantial legacy Python codebases, switching frameworks carries non-trivial migration costs. MONAI provides a mature library of ready-made components—pretrained segmentation networks, standardized data-loading pipelines, and extensive augmentation transforms—that can be deployed with minimal custom code. Similarly, SimpleITK offers a comprehensive catalogue of classical filters whose correctness has been validated across decades. When the research question can be answered with an existing pipeline and the primary requirement is rapid prototyping within a specific tensor framework (like PyTorch), these tools remain highly efficient.

When MedImages.jl offers a clear advantage for SciML. The Julia-based approach becomes compelling when research transcends standard convolutional network training and enters the realm of physics-informed models or custom numerical workflows [7]:

- **Unifying the Fractured SciML Landscape.** In Python, moving a differentiable model from PyTorch to a JAX-based differential equation solver often requires significant code rewriting or bridging overhead. Julia’s multiple dispatch solves the expression problem,

meaning essentially all packages are interoperable natively [5,6]. Med-
 Images.jl serves as the critical entry point to make this uniquely
 unified SciML ecosystem accessible for medical imaging.

- **Novel algorithm development.** When researchers need to imple-
 ment a new spatial transformation, loss function, or physics-informed
 model from scratch, Julia’s single-language stack eliminates the need
 to write C++/CUDA extensions. The algorithm can be prototyped,
 debugged, and GPU-accelerated within the same language, with full
 automatic differentiation support.
- **Cross-domain scientific computing.** Julia’s ecosystem uniquely
 bridges medical imaging with adjacent scientific domains—DifferentialEquations.jl
 for pharmacokinetic modelling, KomaMRI.jl [12] for MRI pulse-
 sequence simulation, Flux.jl/Lux.jl [13] for deep learning—all within
 a single runtime. This composability enables workflows that would
 otherwise require fragile inter-process communication between Python,
 MATLAB, and C++ components.
- **Differentiable imaging physics.** Tasks such as learned registra-
 tion, differentiable augmentation, or physics-informed neural networks
 require gradients to flow through spatial transformations. MedIm-
 ages.jl’s native Julia kernels integrate directly with Zygote.jl and
 Enzyme.jl, whereas MONAI and TorchIO must fall back on non-
 differentiable C++ backends for many geometric operations.

MedImages.jl offers a path forward—a future where imaging physics
 is integrated directly into the deep learning pipeline, where hardware
 acceleration is transparent and ubiquitous, and where the metadata that
 defines our physical world is preserved with the same rigor as the pixel
 values themselves. By bridging the divide between the physics of acquisition
 and the logic of inference, and by addressing the entwined challenges of
Volume and **Speed** with a $7.2\times$ faster turnaround than traditional caching
 frameworks [10], MedImages.jl empowers researchers and clinicians to unlock
 the full potential of medical imaging as a precise, quantitative science.
 This performance is especially critical for large-scale multi-center studies
 and meta-analyses, where the ability to process thousands of subjects
 with absolute metadata integrity enables research that was previously
 computationally infeasible.

Limitations

While MedImages.jl offers significant advantages in execution speed and
 differentiability, adopting the Julia ecosystem presents certain trade-offs.
 The most notable is the “time-to-first-plot” or compilation latency. Because
 Julia uses Just-In-Time compilation, the first execution of a complex spatial
 transformation incurs a compilation penalty, making initial startup slower
 than equivalent interpreted Python code. Furthermore, running full test
 suites or deploying models with heavy dependencies (e.g., CUDA.jl, Lux.jl,
 Zygote.jl) requires significant development memory, occasionally causing
 out-of-memory (OOM) issues on constrained hardware. Finally, while the
 Julia machine learning ecosystem is growing rapidly, it does not yet match
 the sheer volume of pre-trained models and plug-and-play architectural
 components available in the PyTorch/MONAI ecosystem.

5 Conclusions

The development of MedImages.jl addresses core computational and meta-data challenges in modern medical imaging. By leveraging Julia’s compilation architecture and type system, the library provides a unified framework where physical accuracy, computational performance, and end-to-end differentiability are achieved simultaneously. The native implementation of spatial transformations allows for transparent integration with SciML pipelines, offering a robust foundation for the next generation of physics-informed models in molecular imaging and quantitative theranostics. In practice, we envision a complementary workflow: established MONAI/SimpleITK pipelines for routine clinical deployment, and MedImages.jl for research frontiers where performance, differentiability, or algorithmic novelty are the primary requirements.

Availability and Future Directions

MedImages.jl is open-source and available at <https://github.com/JuliaHealth/MedImages.jl>. All experiments described in this paper, including performance benchmarks and differentiability validations, are reproducible via the scripts provided in the repository’s `experiment` directory.

Future work will focus on:

1. Integrating with advanced reconstruction frameworks (KomaMRI.jl [12], GIRFReco.jl [14]) for end-to-end physics-informed learning.
2. Implementing differentiable registration algorithms leveraging Zygote.jl.
3. Expanding support for standardized formats like DICOM-SEG via Highdicom [15] integration.
4. Developing architectural foundations for clinical-grade AI agents (e.g., VoxelPrompt) capable of multi-modal ”Medical Superintelligence” benchmarks where absolute metadata integrity is a prerequisite for safe automated diagnostics.

Declarations

Ethics approval and consent to participate: This research relies solely on retrospective analysis of fully anonymized, publicly available datasets. No new human subjects data was collected. Therefore, formal ethics committee approval and informed consent were not required.

Declaration of competing interests: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding sources: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies: During the preparation of this work, the author(s) used generative AI and AI-assisted technologies for improving style and assisting in code development. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- [1] B. C. Lowekamp, D. T. Chen, L. Ibáñez, D. Blezek, The design of simpleitk, *Frontiers in Neuroinformatics* 7 (2013). doi:10.3389/fninf.2013.00045. URL <http://dx.doi.org/10.3389/fninf.2013.00045>
- [2] M. J. Cardoso, W. Li, R. Brown, N. Ma, E. Kerfoot, Y. Wang, B. Murrey, A. Myronenko, C. Zhao, D. Yang, V. Nath, Y. He, Z. Xu, A. Hatamizadeh, A. Myronenko, W. Zhu, Y. Liu, M. Zheng, Y. Tang, I. Yang, M. Zephyr, B. Hashemian, S. Alle, M. Z. Darestani, C. Budd, M. Modat, T. Vercauteren, G. Wang, Y. Li, Y. Hu, Y. Fu, B. Gorman, H. Johnson, B. Genereaux, B. S. Erdal, V. Gupta, A. Diaz-Pinto, A. Dourson, L. Maier-Hein, P. F. Jaeger, M. Baumgartner, J. Kalpathy-Cramer, M. Flores, J. Kirby, L. A. D. Cooper, H. R. Roth, D. Xu, D. Bericat, R. Floca, S. K. Zhou, H. Shuaib, K. Farahani, K. H. Maier-Hein, S. Aylward, P. Dogra, S. Ourselin, A. Feng, Monai: An open-source framework for deep learning in healthcare, *arXiv* (2022). doi:10.48550/ARXIV.2211.02701. URL <https://arxiv.org/abs/2211.02701>
- [3] J. Dextl, S. Gatidis, M. Früh, K. Jeblick, A. Mittermeier, A. Stüber, B. Schachtner, J. Topalis, M. Fabritius, S. Gu, g. k. Murugesan, J. VanOss, J. Ye, J. He, A. Alloula, B. Papież, Z. Mesbah, R. Modzelewski, M. Hadlich, T. Küstner, Autopet challenge on fully automated lesion segmentation in oncologic pet/ct imaging, part 2: Domain generalization, *Journal of Nuclear Medicine* 67 (2025) jnumed.125.270260. doi:10.2967/jnumed.125.270260.
- [4] J. Bezanson, J. Chen, B. Chung, S. Karpinski, V. B. Shah, J. Vitek, L. Zoubitzky, Julia: dynamism and performance reconciled by design, *Proceedings of the ACM on Programming Languages* 2 (10 2018). doi:10.1145/3276490. URL <http://dx.doi.org/10.1145/3276490>
- [5] S. Pal, M. Bhattacharya, S. Dash, S.-S. Lee, C. Chakraborty, A next-generation dynamic programming language julia: Its features and applications in biological science, *Journal of Advanced Research* 64 (10 2024). doi:10.1016/j.jare.2023.11.015. URL <http://dx.doi.org/10.1016/j.jare.2023.11.015>
- [6] J. Eschle, T. Gál, M. Giordano, P. Gras, B. Hegner, L. Heinrich, U. H. Acosta, S. Kluth, J. Ling, P. Mato, M. Mikhasenko, A. M. Briceño, J. Pivarski, K. Samaras-Tsakiris, O. Schulz, G. A. Stewart, J. Strube, V. Vassilev, Potential of the julia programming language for high energy physics computing, *Computing and Software for Big Science* 7 (10 2023). doi:10.1007/s41781-023-00104-x. URL <http://dx.doi.org/10.1007/s41781-023-00104-x>
- [7] E. Roesch, J. G. Greener, A. L. MacLean, H. Nassar, C. Rackauckas, T. E. Holy, M. P. H. Stumpf, Julia for biologists, *Nature Methods* 20 (4 2023). doi:10.1038/s41592-023-01832-z. URL <http://dx.doi.org/10.1038/s41592-023-01832-z>
- [8] M. Innes, A. Edelman, K. Fischer, C. Rackauckus, E. Saba, V. Shah, W. Tebbutt, Zygote: A differentiable programming system to bridge

- machine learning and scientific computing (07 2019). doi:10.48550/arXiv.1907.07587. 644
645
- [9] E. Berman, J. Ginesin, The state of julia for scientific machine learning 646
(10 2024). doi:10.48550/arXiv.2410.10908. 647
- [10] J. Shi, H. Ghazzai, Y. Massoud, Differentiable image data augmentation 648
and its applications: A survey, IEEE Transactions on Pattern Analysis
and Machine Intelligence 46 (2023) 1148 – 1164. doi:10.1109/TPAMI.
2023.3330862. 650
651
- [11] O. Esteban, C. Markiewicz, R. Blair, C. Moodie, A. Isik, A. Aliaga, 652
J. Kent, M. Goncalves, E. DuPre, M. Snyder, H. Oya, S. Ghosh,
J. Wright, J. Durnez, R. Poldrack, K. Gorgolewski, Fmriprep: a robust 653
preprocessing pipeline for functional mri (04 2018). doi:10.1101/
306951. 654
655
656
- [12] C. Castillo-Passi, R. Coronado, G. Varela-Mattatall, C. Alberola-López, 657
R. Botnar, P. Irarrazaval, Komamri.jl: An open-source framework for
general mri simulations with gpu acceleration, Magnetic Resonance in 658
Medicine 90 (3 2023). doi:10.1002/mrm.29635. 659
URL <http://dx.doi.org/10.1002/mrm.29635> 660
661
- [13] A. Pal, Lux: Explicit Parameterization of Deep Neural Networks in 662
Julia, if you use this software, please cite it as below. (Apr. 2023).
doi:10.5281/zenodo.7808903. 663
URL <https://doi.org/10.5281/zenodo.7808903> 664
665
- [14] A. Jaffray, Z. Wu, S. J. Vannesjo, K. Uludağ, L. Kasper, Girfreco.jl: 666
An open-source pipeline for spiral magnetic resonance image (mri)
reconstruction in julia, Journal of Open Source Software 9 (5 2024). 667
doi:10.21105/joss.05877. 668
URL <http://dx.doi.org/10.21105/joss.05877> 669
670
- [15] C. P. Bridge, C. Gorman, S. Pieper, S. W. Doyle, J. K. Lennerz, 671
J. Kalpathy-Cramer, D. A. Clunie, A. Y. Fedorov, M. D. Her-
rmann, Highdicom: a python library for standardized encoding of 672
image annotations and machine learning model outputs in pathol- 673
ogy and radiology, Journal of Digital Imaging 35 (8 2022). doi: 674
10.1007/s10278-022-00683-y. 675
URL <http://dx.doi.org/10.1007/s10278-022-00683-y> 676
677